

Shared UI State and Host Enforcement: A Bounded Case Study of Three Agent-UI Configurations

Eduardo Arana

Arananet

21 September 2026 — local draft; human review pending

Abstract

Agent-facing UI libraries and protocols provide different ways to describe, host, and interact with interfaces. Comparing them requires separating the format from the adapter and the host that implements it. We examine an existing deterministic TypeScript harness using json-render, A2UI, and MCP Apps in three scenarios: surface handoff, fixed-order identifier collision, and action round-trip. The shared-state configurations retain two handoff blocks in one surface and replace an earlier summary when a later writer uses the same identifier. The MCP Apps configuration allocates separate instances and retains both summaries. Its model-only tool rejection depends on an explicitly installed host handler; the existing permissive variant forwards the call. These observations concern the tested configurations, not intrinsic rankings of protocols. We map claims to source, assertions, and execution evidence, distinguish retained artifacts from fresh reruns, and identify confounders and missing checks. The local protocol suite passed 24 tests; fresh browser execution was blocked by an environment failure documented with the artifacts. This is a retrospective engineering case study, with no LLM experiment, performance measurement, or independent-person reproduction. Human review remains pending.

1 Introduction

A UI description, an execution environment, and an authorization decision operate at different layers. Treating them as interchangeable can make a comparison attribute a host policy to a wire format, or attribute a namespace decision to a universal limitation. The practical question is what the current configurations do when more than one named writer contributes to a conceptual interface.

The contribution is an auditable interpretation of a small existing harness. We provide an explicit scope, a falsifiable hypothesis, a claim-to-check mapping, and a reproduction procedure. We do not claim exhaustive novelty, replication of a prior study, an impossibility proof, a fundamental tradeoff, or a universal protocol ranking. The retained comparison report is an input to audit, not a scientific oracle. Its broader wording and historical scenario names remain unchanged; the bounds in this manuscript govern our conclusions.

2 Research questions

RQ1: Where do the existing adapters place the output of a scripted surface handoff?

RQ2: What survives a fixed-order identifier-collision probe in the configured namespaces?

RQ3: Which local bookkeeping and host decisions determine action routing and model-only tool rejection?

3 Methodology

This retrospective case study examines deterministic scripted agents, with no LLM experiment. The unit of comparison is an SDK/adapter/host configuration. Identical scenario input does not isolate protocol causality. Three scenarios exercise three configurations, with an existing permissive-host variant for MCP Apps in S3.

3.1 A falsifiable local hypothesis

H1: In S2, the fixed-order writes to the shared `summary` ID leave the finance headline in the `json-render` and `A2UI` configurations, while the MCP Apps configuration retains both headlines in separate instances. A snapshot or DOM assertion showing the risk headline instead of finance in a shared summary, or loss of either isolated headline, would contradict this hypothesis for the tested configuration.

The S2 script awaits each emission: risk detail, finance detail, risk summary, then finance summary. This is a fixed-order identifier-collision probe, not actual concurrency or a scheduling benchmark. The historical filename and test descriptions retain the word “concurrent”. The focused check uses the existing snapshot assertions and browser assertions, rather than treating the adapter’s `LOST` or `ENFORCED` labels as measurements. Fresh execution status is recorded separately from this source-based hypothesis.

3.2 Configurations and versions

The manifest specifies ranges. The lockfile and locally installed package metadata agree on the exact versions below. Package versions are not protocol schema versions. These are observations of this checkout, not recommendations or statements about the newest available releases.

Configuration	Manifest range	Lockfile and installed version
<code>json-render</code> core and <code>React</code> renderer	<code>^0.21.0</code> each	<code>0.21.0</code> each
<code>A2UI</code> web core and <code>Lit</code> renderer	<code>^0.11.0</code> each	<code>0.11.0</code> each
MCP Apps extension SDK	<code>^2.0.0</code>	<code>2.0.0</code>
MCP client, core, and server	<code>^2.0.0</code> each	<code>2.0.0</code> each
<code>React</code> and <code>React</code> DOM	<code>^19.2.3</code> each	<code>19.3.0</code> each
<code>Lit</code>	<code>^3.3.3</code>	<code>3.3.3</code>
<code>Zod</code>	<code>^4.2.0</code>	<code>4.6.5</code>

The `A2UI` adapter imports `@a2ui/web_core/v0_9` and emits `version: "v0.9"`. Its package version `0.11.0` does not mean that the experiment uses a `v0.11` protocol. MCP Apps SDK `2.0.0` likewise does not establish a wire protocol version. The extension specification and SDK are separate references; no exact upstream specification commit or negotiated wire-version value was retained

for this paper. Live repository documentation was inspected on 21 September 2026 and may differ from release-time documentation. The local lockfile fixes the tested package resolution, not the mutable web references.

The json-render adapter maintains one `Spec` per conceptual surface and maps logical block, row, and control IDs to element keys. It calls `applySpecPatch` for element additions/replacements and also directly updates root children. Therefore, its patch log does not capture every mutation. Its catalog validates emitted specs; `writtenBy` is an ordinary catalog prop and `lastWriter` is adapter bookkeeping. Neither is authenticated provenance.

The A2UI adapter validates messages using `A2uiMessageSchema` and drives `MessageProcessor`. Its component IDs and data paths derive from block IDs. A shared `rootChildren` collection coordinates the surface root. Node uses a schema-only harness catalog; the browser uses the Lit basic catalog. Snapshot titles and writer names partly come from adapter maps, while row values come from the data model. Surface addressing is not authorization: an address identifies a target but does not establish a caller's permission.

The MCP Apps adapter creates one server per agent and one app instance per surface/agent pair. Its Node path uses real SDK clients, servers, apps, and bridges connected by in-memory transports. Its snapshot is reconstructed from adapter-held blocks, not from a browser DOM. The browser host mounts an iframe with `sandbox="allow-scripts"` for each agent and uses `PostMessageTransport`. This topology is an implementation choice. Connection identity is not universally agent identity: the mapping works here because the host establishes one server per named agent. A server could aggregate multiple agents in a different application.

Figure 1 locates the shared state, local ownership mapping, and enforcing handler in these configurations. The dashed path denotes the existing permissive variant, not a second enforcement mechanism.

3.3 Scenario design and evidence collection

S1 opens a trip surface, emits a planner itinerary, hands off to booking, and emits a reservation. S2 performs the fixed-order writes specified in H1. S3 emits basket and payment controls and activates ordinary and privileged actions. The amounts and booking operations are fixtures: the MCP server records calls and returns text; it does not charge money. The three scenarios are fixed examples, not a sample drawn from a population of agent workflows.

The runner constructs fresh adapters per scenario and configuration. The orchestrator is a scripted stand-in with named agents and a control-to-agent routing table, not an autonomous model-driven system. We inspect the existing tests and rerun them where local prerequisites permit; no scenarios or adapters are added or modified.

The report generator takes adapter-authored classifications and uses first-recording-wins selection for each capability and configuration. It does not independently measure the meaning of `SUPPORTED`, `LOST`, `ENFORCED`, or `NOT_EXPRESSIBLE`. Repeated labels do not constitute independent observations. Timestamp normalization makes its Markdown stable but does not validate its interpretation. A freshness check establishes correspondence between report and generator, not the truth of every sentence in the report.

We therefore distinguish classifications from assertions on snapshots, messages, tool responses/refusals, and rendered elements. The conformance tests validate the S1 A2UI messages and json-render catalog output, examine recorded patches, and check report and image-link freshness. They do not establish full specification conformance. The A2UI deletion message is not exercised in that S1 check. The json-render patch assertions do not cover its direct root mutation.

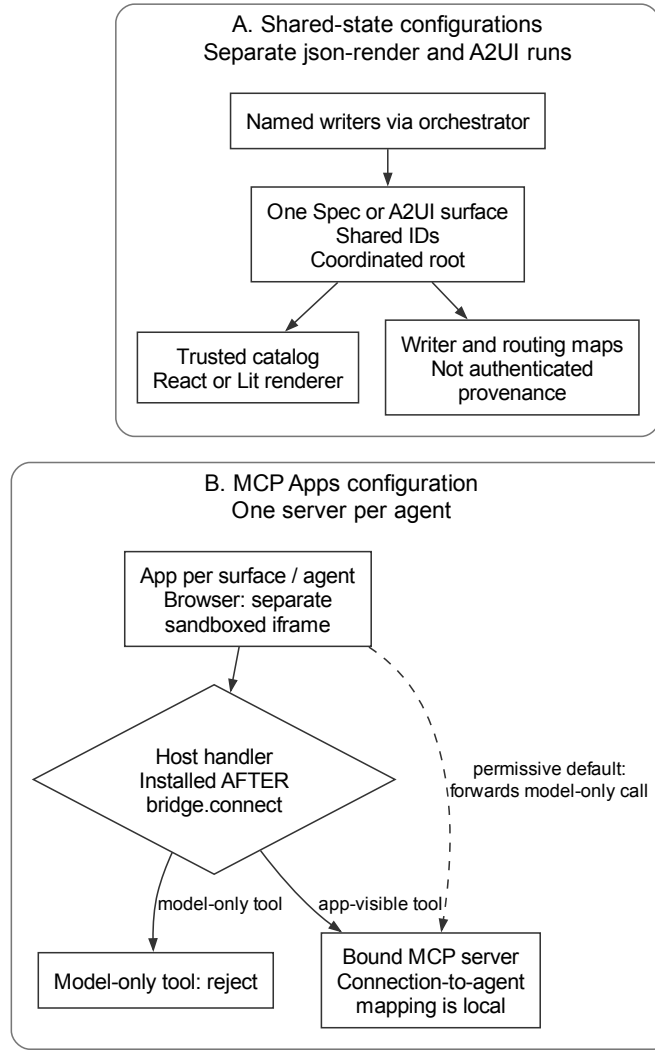


Figure 1: Configured state and enforcement boundaries. Panel A summarizes two separate declarative-renderer runs; panel B shows the app-to-server call path for each MCP instance. Writer maps are bookkeeping. Only the installed host handler rejects the model-only call; the permissive default forwards it. Original source-derived illustration, not a fresh browser observation.

Browser evidence has an additional qualification: the MCP browser host contains its own fixture script. In S2 it renders the two summaries without the two uncontested detail blocks used by the Node scenario. Its DOM assertions are useful corroboration for those summaries, not byte-identical replay of the Node runner. Retained screenshots alone cannot establish that a fresh browser run passed.

4 Bounded observed results

4.1 RQ1: Handoff and placement

Existing S1 assertions check a single `trip` snapshot region containing both `itinerary` and `reservation` in `json-render` and A2UI. MCP Apps checks separate `trip::planner` and `trip::booking` regions. The browser assertions check both blocks within one React tree or A2UI surface, and separate MCP iframes. These checks support a placement observation under the chosen mappings. They do not prove that MCP Apps cannot represent an aggregated UI or that either declarative format provides authenticated cross-agent ownership.

4.2 RQ2: Fixed-order identifier collision

The focused local S2 rerun passed its three selected tests; ten non-S2 tests in that file were skipped by the filter. The `json-render` assertion checks the finance writer and headline, plus survival of the two uncontested blocks. The A2UI Node assertion checks the finance writer from adapter bookkeeping; its browser assertion checks headline text. MCP snapshot assertions check both headlines in different instances. H1 is supported by these local assertions to their respective extent; A2UI's Node writer field is weaker evidence than a rendered headline. Fresh browser execution was blocked before tests could run because the preview server could not listen inside the sandbox. We therefore do not count its DOM expectations as newly executed evidence.

The replacement is consistent with reusing a slot in these shared namespaces. It is not an observed scheduling race. The absence of a protocol-level conflict notification in this script does not mean the harness is blind to replacement: the adapter itself detects a changed writer and records it. Namespacing, ownership checks, arbitration, or shared aggregation could change the outcome; none is evaluated as an intervention here.

Figure 2 separates the four awaited writes from the retained summary states. The arrows express script order, not elapsed time or competing schedules.

4.3 RQ3: Routing and host decisions

In the shared-state configurations, ordinary action routing uses the orchestrator's control table. Tests inspect routing entries and deliveries, and the A2UI action envelope carries surface/component fields and a timestamp without an agent field. These payload observations do not rule out an application-supplied identity envelope. Encoding a name in context or props would still require trusted binding before it could authorize anything.

`json-render`'s privileged control includes `confirm` metadata. The Node adapter constructs an event and classifies that metadata; it does not run an independent authorization service. The browser suite clicks the ordinary `json-render` action, not the privileged confirmation flow. Confirmation UI is not independent authorization, and the existing assertions do not establish complete consent behavior or protection against an omitted confirmation binding. A2UI has no privileged gate implemented in this adapter; that is not a proof that its hosts or trusted catalog components cannot impose one.

For MCP Apps, the host installs its enforcing `oncalltool` handler after `bridge.connect`. The handler looks up the tool and rejects model-only visibility. The bridge tests check successful app-visible calls, rejection messages and refusal records, and successful model-only calls when enforcement is disabled. The permissive host variant forwards that call. This is evidence about the tested

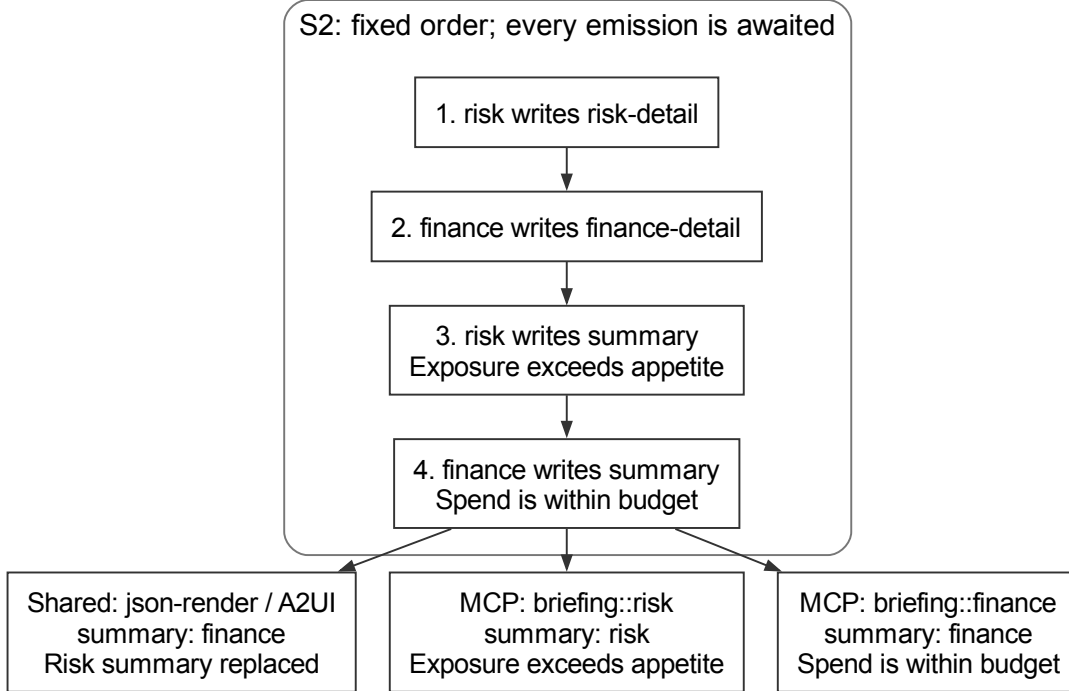


Figure 2: S2 write order and retained summaries in the configured namespaces. The uncontested detail writes precede the collision. Node assertions check the json-render headline and both MCP headlines; A2UI’s Node check verifies the last-writer map, while its headline DOM assertion remains unexecuted locally. The two MCP summaries belong to separate instances, not a merged surface.

SDK and handler installation order, not every MCP host or SDK release. The inspected extension specification assigns visibility enforcement to the host [R4].

Routing evidence is narrower than some trace labels suggest. For actions, the Node MCP adapter selects the first instance for a conceptual surface. Its helper is `findInstanceForSurface`. It does not find the agent owning the clicked control. The routing test only requires some delivery without the fallback table. Thus it cannot prove correct owner routing for every S3 control. Browser S3 instead clicks inside the cart and payments frames separately. We retain this distinction and leave the existing behavior unchanged.

5 Claim-evidence mapping

Here **supported** means a bounded statement has a matching inspected assertion or direct source observation, with run status separately recorded. **Partial** means a relevant assertion exists but leaves a material inference untested. **Proposed** means a future check has not been performed. These terms are this paper’s evidence statuses, distinct from the adapter outcome vocabulary.

Claim	Status	Evidence and falsifier
C1: S1 yields one shared region or two MCP instances in this topology.	Supported	S1 snapshot region and block assertions; contradicted by missing blocks or different region counts.
C2: S2 overwrites the json-render summary and retains both MCP summaries.	Supported	S2 headline assertions; contradicted by different retained values.
C3: S2’s A2UI shared summary displays finance’s headline.	Partial	Node checks adapter writer; browser checks rendered value. Fresh DOM execution remains a separate prerequisite.
C4: S2 is sequential, with no scheduling test.	Supported	Four awaited emissions in the S2 source; contradicted by an overlapping scheduler in the executed path.
C5: The enforcing MCP handler rejects the model-only call, while the permissive variant forwards it.	Supported	Bridge rejection/refusal and successful-result assertions; contradicted by reversed or identical behavior.
C6: Every MCP S3 control reaches its intended agent.	Partial	Existing test checks some table-free delivery; first-instance selection prevents this stronger inference.
C7: json-render includes confirmation metadata, with consent enforcement incompletely checked.	Partial	Existing check records metadata, not independent policy or privileged browser consent.
C8: Matrix labels are adapter classifications, not independent measurements.	Supported	Source shows first-recording-wins selection; freshness checks correspondence only.
C9: A matched-topology intervention isolates protocol causality.	Proposed	Pending controlled comparison; no new experiment is reported.

The linked scenario, bridge, conformance, and browser test files in Methodology and Results are the check locations for C1–C8. This mapping exposes uneven coverage rather than assigning a numeric score across unlike checks.

6 Related work

The json-render repository describes catalog-defined components and actions rendered from structured specifications [R1]. A2UI similarly separates declarative UI data from client implementation and describes a trusted component catalog [R2]. These are meaningful security properties: constraining requested components and handlers can limit what untrusted UI data asks the client to run. They do not by themselves establish per-writer authorization in our shared namespace. Conversely, our collision example is not evidence of blanket insecurity in declarative rendering.

The MCP Apps repository describes tool-associated UI resources and a host bridge [R3]. Its specification describes visibility checks and sandbox/CSP obligations [R4]. Those obligations are not proof that this harness checks all of them. In particular, iframe sandbox attributes and successful bridge tests do not establish exhaustive CSP enforcement, transport security, or resistance to

malicious content. Declarative catalogs and sandboxed app delivery are compatible layers: a hosted application could use a declarative renderer internally. No integration experiment is reported here.

The requested methodological reference is besanson/sarc-authority-derivation. Access attempts for its README, REPRODUCTION.md, NOVELTY.md, and paper directory failed in this environment. We attribute the intended methodological influence explicitly, but cannot claim direct inspection of its manuscript or verification of its results. The user’s description of that project’s practices motivated our research questions, claim table, status vocabulary, and execution ledger. No prose or scientific results from that project are used as evidence here. Any reproduction credited there to Eduardo concerns that project only. The present work is neither replication of it nor an independent-person reproduction. The reference inspection remains pending, and this related-work discussion is deliberately selective rather than an exhaustive literature or novelty review.

7 Threats to validity

Internal validity. Identical logical inputs leave adapter bookkeeping, shared-root coordination, naming conventions, host enforcement, catalogs, rendering frameworks, transports, and one-server-per-agent MCP topology as confounders. The default/permissive MCP comparison narrows one host-policy question, but it is not a factorial comparison across all configurations.

Construct validity. A region count is a property of the adapter snapshot; it is not a usability measure. A writer prop is not authenticated provenance. Addressability is not authority. A confirmation binding is not an authorization decision. Adapter labels and some scenario assertions share their interpretation, creating a risk of circular validation. The independent aspects of the checks are the inspected contents, response/refusal assertions, and DOM expectations; they are still authored within this repository, not by independent evaluators.

Execution and coverage. Node transports do not test browser isolation. The MCP browser fixtures differ from the Node scenario. Existing screenshots are retained artifacts with no newly established capture provenance. The suite does not comprehensively test adversarial inputs, identity forgery, transport authentication, CSP, cross-server policy, authorization revocation, or arbitrary concurrent schedules. No completed security review is claimed.

External and temporal validity. Three designed scenarios, fixed fixture data, and the pinned installed packages cannot establish population effects or general protocol superiority. Later SDKs, other renderers, different server topologies, and stricter application policies can behave differently. Mutable upstream documentation is contextual evidence, not a version-pinned release archive. No latency, throughput, cost, user study, or model capability was measured.

8 Reproducibility

The reproduction guide gives commands and expected semantic outcomes. The local execution record separates successful commands, failures, unavailable prerequisites, and retained artifacts. The base revision is `b895e57671e118fcd3a7f86892fc1b2526b78c9c`; the initial working tree was clean. The manuscript, generated derivatives, checks, and documentation additions are local uncommitted work on that base. Neither the historical revision supplied as a hint nor a clean base revision alone identifies the final paper workspace. The execution record and file hashes identify the relevant inputs and outputs.

Semantic reproducibility means obtaining the specified regions, values, events, and refusal behavior under the same configuration. Byte identity is a separate question. A2UI action timestamps

vary; report generation normalizes those timestamps. PDF metadata and toolchain resources can also vary. We check generated LaTeX against fresh Pandoc output under the recorded toolchain, not PDF byte identity across machines. The Markdown manuscript is the only editable paper source; LaTeX and PDF are generated derivatives.

This session is an agent-assisted local rerun of existing checks. It establishes neither independent-person reproduction nor human approval. The previous session’s reported dependency installation is not a fresh `npm ci` execution in this record. Missing checks and failed attempts remain visible rather than being counted as passes. OpenSpec command success cannot prove every acceptance criterion.

The complete local Vitest run passed 24 tests across three files and TypeScript checking passed. The standalone `report:check` command failed when `tsx` attempted to create an IPC listener (EPERM); the report freshness assertion inside the passing Vitest suite is separate evidence, not a successful standalone command. The browser command built the pages but failed to start its preview server (EPERM on `::1:5178`), so no fresh browser test success is claimed. Pandoc produced LaTeX and its freshness check passed. The initial Tectonic invocation panicked internally (exit 101) in cached-only mode. A later local build used an explicit directory bundle assembled from already cached TeX resources and a smaller template, and produced the illustrated PDF without network access or shell escape. Original Mermaid diagrams were converted locally to vector figures through a restricted Mermaid-to-Graphviz renderer after Chromium-based rendering failed. These illustrations explain source and assertions; they add no experiments. The execution record retains the historical failures and current inspection evidence. Local compilation does not establish arXiv TeX Live compatibility or publication acceptance. Human review remains pending.

9 Conclusion

For the inspected configurations, handoff placement and repeated identifiers follow the chosen shared or separated namespaces, while MCP model-only tool rejection depends on the enforcing host handler. The strongest local conclusion is that claims must name the adapter mapping and enforcement site alongside the SDK or protocol. The results do not establish unavoidable incompatibility between composition and security. Integrations remain possible, and trusted catalogs remain relevant security mechanisms.

Future experiments are explicitly pending: matched namespaces and topology, controlled scheduling with overlapping writes, trusted identity envelopes, privileged confirmation interaction tests, and complete host-policy tests. They are suggestions for separate approved work, not results of this paper.

10 AI-assistance disclosure

Eduardo Arana (Arananet) is the named author at the user’s request. An OpenAI Codex coding assistant inspected source and available references, drafted this English manuscript and supporting documentation, added focused checks, and executed local validation commands. This disclosure does not assert that Eduardo personally performed each command, reviewed every claim, or approved publication. Human review pending: the author must assess scientific accuracy, citations, limitations, and final presentation. AI-assisted drafting is separate from the deterministic harness: no LLM participates in its scenario experiments. No model identifier is inferred from an invocation request or catalog description.

11 References

R1. Vercel Labs. `json-render` repository and README. Inspected 21 September 2026; live main, upstream commit not retained. Local SDK context: `@json-render/core` and `@json-render/react` 0.21.0, including the installed core README and manifest. Used for catalog/renderer design, not a security proof.

R2. A2UI project. A2UI repository and README (the inspected `google/A2UI` URL redirects here). Inspected 21 September 2026; live main, upstream commit not retained. Local context: web core and Lit 0.11.0, installed web-core README, and adapter-selected v0.9 schema.

R3. Model Context Protocol contributors. MCP Apps SDK repository and README. Inspected 21 September 2026; live main, upstream commit not retained. Local context: extension, client, core, and server packages 2.0.0; extension README and manifests inspected. SDK package version is distinct from wire version.

R4. Model Context Protocol contributors. MCP Apps extension specification, `draft/apps.mdx`. Visibility and sandbox/CSP sections inspected 21 September 2026; mutable main path, no pinned specification commit. Used to distinguish host requirements from the assertions implemented here. The filename’s “draft” is retained as the source location, without claiming a release status from the SDK version.